`sqisign-selkie`: Implementing SQIsign v2 NIST-I in Rust

Deirdre Connolly

Selkie Cryptography, USA

Abstract. SQIsign has the smallest signatures and public keys of any post-quantum signature scheme. It is also one of the hardest to implement correctly. We describe `sqisign-selkie`, a Rust implementation of SQIsign v2 for NIST-I, built from the specification and C reference implementation. Along the way we found eight bugs in our theta-coordinate isogeny chain, each traceable to an ambiguity in the specification or an implicit convention in the reference code. We catalog these bugs, explain how they arose, and extract concrete lessons for anyone implementing isogeny-based schemes from a spec. We show how Rust's type system enforces cryptographic invariants at compile time, and survey the published constant-time techniques for SQIsign's quaternion arithmetic.

Keywords: SQIsign · isogeny-based cryptography · Rust · constant-time · post-quantum signatures

1 Introduction

SQIsign [SQI25] is the only isogeny-based signature scheme in Round 2 of NIST's additional signature standardization process. Its 148-byte signatures and 65-byte public keys at NIST-I are smaller than any other post-quantum scheme under consideration. But that compactness comes at a cost: implementing SQIsign requires quaternion algebras, the Deuring correspondence, dimension-2 theta-coordinate isogenies, and a signing algorithm that touches all of them.

The specification [SQI25] is mathematically precise but leaves many implementation choices unstated. The C reference implementation fills in those choices—but they are embedded in code, not documented. An implementor working from the spec alone will make wrong guesses. We know because we did.

This paper documents `sqisign-selkie`, a Rust library for the NIST-I parameter set ($p = 5 \cdot 2^{248} - 1$) only. By fixing the parameter set at compile time, we specialize aggressively: field arithmetic uses a radix-51 Montgomery representation tuned to this prime, integer widths match proven worst-case bounds for NIST-I quaternion intermediates [KLKL25], and isogeny degree types are sized to the 246-bit maximum that NIST-I permits. We do not attempt to be generic over parameter sets.

We target four goals:

1. **Interoperability.** Verify and produce signatures that the C reference implementation accepts, byte-for-byte, against the NIST KAT vectors.
2. **Misuse resistance.** If a value is always positive and odd, it should be impossible to construct one that isn't. If a lattice operation requires Hermite Normal Form, the compiler should reject non-HNF input. This protects not just end-users of the public API, but us writing the internals, and anyone who maintains this code after us.

E-mail: durumcrustulum@gmail.com (Deirdre Connolly)

3. **Constant-time signing.** The specification does not discuss side channels. The C reference is variable-time for the entire quaternion layer. We traced the signing algorithm and confirmed that the quaternion operations compute over secret-key-derived data. Published attacks [MCJ+25] demonstrate practical SPA on Cornacchia's algorithm. Constant-time execution is not a nice-to-have; it is a requirement.

4. **Idiomatic Rust.** Methods on types over free functions. Invariants via distinct types, not runtime checks. Code that a Rust developer would recognize as natural [Rus24].

This paper is a living document, updated as the implementation progresses.

Contributions.

- A catalog of eight theta-coordinate bugs (§4) with root causes and lessons.
- Compile-time invariant enforcement via Rust's type system (§5).
- A survey of constant-time SQIsign techniques and a concrete hardening plan (§6).

2 Background

We assume familiarity with elliptic curves and isogenies at the level of the SQIsign specification [SQI25], but not with the internals of theta-coordinate arithmetic or quaternion ideal algorithms. Where the spec is unclear, we say so; where the C reference makes an undocumented choice, we document it here.

3 Architecture

The implementation is organized into five layers:

`fields/` Prime field $\mathbb{F}_p$ (radix-51 Montgomery form) and quadratic extension $\mathbb{F}_{p^2}$.

`curves/` Montgomery curves, x-only projective arithmetic, torsion bases, 2^e-isogeny chains, and the `IsogenyDegree` newtype.

`surfaces/` Abelian surfaces in theta coordinates, $(2,2)$-isogeny chains (Algorithm 8.47), gluing, and splitting.

`quaternions/` Quaternion algebra $B_{p,\infty}$, lattices, ideals, HNF, L2 reduction, and `RepresentInteger`.

`deuring/` The Deuring correspondence bridge: `IdealToIsogeny`, `FixedDegreeIsogeny`, `SuitableIdeals`, action matrices.

4 Bugs from Specification Ambiguity

During KAT verification testing against the C reference implementation, we discovered multiple bugs in our theta-coordinate $(2,2)$-isogeny chain. Each bug originated from an ambiguity in the specification or an implicit convention in the reference code. We catalog them here with their root causes.

4.1 ProductToTheta coordinate convention

The spec describes the product theta structure abstractly via level-2 theta functions. The relationship between Montgomery projective coordinates $(X : Z)$ and the product theta representation is not stated explicitly. Our initial implementation used $(X - Z, X + Z)$ as "theta-like" coordinates—natural for Montgomery differential addition—but the C reference's `base_change()` uses raw (X, Z) directly:

```
fp2_mul(&null.x, &T->P1.x, &T->P2.x); // X1 * X2
fp2_mul(&null.y, &T->P1.x, &T->P2.z); // X1 * Z2
fp2_mul(&null.z, &T->P2.x, &T->P1.z); // Z1 * X2
fp2_mul(&null.t, &T->P1.z, &T->P2.z); // Z1 * Z2
```

Lesson: When the spec is abstract about coordinate representations, the C reference is the ground truth.

4.2 Squared theta means square then Hadamard

In the SQIsign context, "squared theta" means component-wise squaring *followed by a Hadamard transform*. The C reference wraps this in `to_squared_theta()`, whose name does not reveal the Hadamard. Our generic isogeny evaluation (Algorithm 8.34) computed $\mathcal{H}(\text{inv} \cdot P^2)$ instead of $\mathcal{H}(\text{inv} \cdot \mathcal{H}(P^2))$.
Lesson: Read the body of helper functions in the C reference, not just their names. "Squared theta" is not just squaring.

4.3 Cross-product index errors in GluingEval

Algorithm 8.39 computes `ADDComponents` on each curve component, producing (u_1, v_1, w_1) from curve 1 and (u_2, v_2, w_2) from curve 2. The U vector's second entry should be $u_1 \cdot w_2$ (cross-curve), but we wrote $u_1 \cdot w_1$ (same-curve).
Lesson: In product-curve formulas, every multiplication involves one factor from each curve. Two factors with the same subscript are almost certainly wrong.

4.4 Projective inverse vs. field inversion

The gluing codomain needs the "inverse" of the dual theta null point $(\alpha, \beta, \gamma, 0)$. In projective coordinates this is computed via cross-products—pure multiplications, no field inversion. Our code called `Fp2::invert()`, which is both wrong and expensive.
Lesson: In projective or theta coordinates, "inverse" almost always means projective inverse via cross-products. Actual field inversion is a red flag.

4.5 Reusing generic types for special patterns

The image of the kernel generator under gluing has the pattern $(x : x : y : y)$. Storing this in a generic 4-coordinate `JacobianPoint` creates ambiguity: `J.y` equals x (the second copy), not y. The scaling step used `J.x` and `J.y` for the two distinct values, but both hold x.
Lesson: Do not reuse a generic n-coordinate type to store a value with a special pattern. Use a dedicated type, or at minimum document which fields hold which logical values.

4.6 Torsion basis swap convention

The C reference's `ec_curve_to_basis_2f_from_hint()` deliberately stores $P - Q$ in `B.Q` and Q in `B.PmQ`:

```
difference_point(&PQ2->Q, &P, &Q, curve);   // B.Q = P - Q
copy_point(&PQ2->P, &P);                     // B.P = P
copy_point(&PQ2->PmQ, &Q);                   // B.PmQ = Q
```

This means the three-point ladder computes $P + [m](P - Q)$, not $P + [m]Q$. The swap ensures the multiplied point avoids $(0, 0)$ for differential addition. This is not documented in the spec.
Lesson: Undocumented conventions in reference code can look like bugs in your own code. Verify against the reference before "fixing" something that isn't broken.

4.7 Different formulas for the last two chain steps

The C reference uses three different formula variants across the $(2, 2)$-isogeny chain, controlled by two boolean flags `hadamard_bool_1` and `hadamard_bool_2`:

- **Normal steps** ($i < n - 2$): `bool_1=0, bool_2=1`. Codomain is Hadamard-transformed (standard form). Eval computes $\mathcal{H}(\text{precomp} \cdot \mathcal{H}(P^2))$.
- **Penultimate step** ($i = n-2$): `bool_1=0, bool_2=0`. Codomain stays in dual form (no Hadamard). Eval computes $\text{precomp} \cdot \mathcal{H}(P^2)$.
- **Ultimate step** ($i = n - 1$): `bool_1=1, bool_2=0`. Input gets an extra Hadamard before squaring; codomain in dual form. Eval computes $\text{precomp} \cdot \mathcal{H}(\mathcal{H}(P)^2)$.

The splitting step expects its input in dual form—what `bool_2=0` produces. Our code used the normal-step formula (`bool_2=1`) for all steps, feeding the splitting a Hadamard-transformed null point that has no zero $U_{i,j}(0)$ entry. The chain ran correctly for 125 steps and produced nonsense at the end.
Lesson: The last few steps of an isogeny chain often need special handling. The spec describes the generic step uniformly; the `hadamard_bool` optimization is an implementation-level choice in the C reference that avoids redundant Hadamard transforms at the boundary between the chain and the splitting step. Check the loop structure, not just the per-step formula.

4.8 $\mathbb{F}_{p^2}$ square root canonicalization

This was the single most impactful bug we encountered, and we devote extra space to it because its consequences are subtle and far-reaching.

Every nonzero square in $\mathbb{F}_{p^2}$ has exactly two roots, r and $-r$. The SQIsign specification (Algorithm 8.12, the `difference_point` subroutine of `TorsionBasisFromHint`) says "compute a square root" without specifying which one. We initially returned whichever root the Tonelli–Shanks formula produced.

The C reference's `fp2_sqrt` (described in Aardal et al. [AAC+24], §3.1) has a five-line canonicalization step at the end: it negates the output whenever the real part is odd (as an integer in $[0, p)$), or when the real part is zero and the imaginary part is odd. This ensures a unique, deterministic "even" root for every input.

Without this canonicalization, `projective_difference` nondeterministically adds $+\sqrt{\Delta}$ or $-\sqrt{\Delta}$ to B_{XZ}, producing two affinely distinct candidates for $x(P - Q)$. Only one matches the C reference's `difference_point`. The wrong choice propagates through `ladder3pt` (producing a different kernel generator), the challenge isogeny (landing on a different curve E_{chl} with a different j-invariant), and every subsequent step. In our case, the challenge curve's j-invariant was `0x03007c`... instead of the correct `0x026558`...—completely unrelated values, not just a sign flip.

The key insight is that the choice of square root is *not absorbed by projective equivalence*. The formula $x_{P-Q} = (\sqrt{\Delta}+B_{XZ} : B_{ZZ})$ is not projectively equivalent to $(-\sqrt{\Delta}+B_{XZ} : B_{ZZ})$; these are genuinely different points. Any implementation of SQIsign that computes torsion bases via `difference_point` must canonicalize its $\mathbb{F}_{p^2}$ square root.
Lesson: Any time a specification says "compute a square root" in $\mathbb{F}_{p^2}$, the implementation *must* specify and enforce a canonical choice. The even-real-part convention used by the C reference is simple and should be the default for any SQIsign implementation. This is not documented in the SQIsign specification as of v2.0.1.

4.9 Never recompute $P-Q$ via `DifferencePoint`

A torsion basis $(P, Q, P-Q)$ is produced once by `TorsionBasisFromHint`. The third component $P-Q$ is computed via `DifferencePoint`, which takes an $\mathbb{F}_{p^2}$ square root. We discovered—after fixing the sqrt canonicalization—that $P-Q$ must *never* be recomputed from P and Q in any subsequent operation. Even with a canonical sqrt, the two roots of `DifferencePoint` yield the affinely distinct points $x(P-Q)$ and $x(2P-Q)$, and there is no guarantee that a fresh call returns the same one.

We found three separate sites where recomputation caused failure:

1. **After scaling.** We doubled P and Q to reduce their order, then called `projective_difference` on the doubled points instead of doubling the original $P-Q$ alongside.
2. **In the matrix application.** $M_{\text{chl}} \cdot (P, Q)$ produced P' and Q' via two biladder calls. We computed $P'-Q'$ via `projective_difference` instead of a third biladder with scalars $(a-b, c-d)$, as the C reference does in `matrix_scalar_application_even_basis`.
3. **Before the $(2,2)$-chain.** After the even response isogeny, we recomputed $P-Q$ from the images instead of pushing the original $P-Q$ through the isogeny as a third evaluation point.

Each fix was individually necessary; all three together were sufficient to make KAT vector 0 verify.
Lesson: In x-only Montgomery arithmetic, $P-Q$ is a *derived value* that cannot be reconstructed from $x(P)$ and $x(Q)$ alone (because $x(P-Q) = x(P+Q)$ on the Kummer line, and `DifferencePoint` resolves this ambiguity via a square root whose sign is fragile). Treat $P-Q$ as a first-class member of the basis triple and propagate it through every transformation.

5 Type Safety for Cryptographic Invariants

Rust's type system lets us make illegal states unrepresentable. Each of the following types prevents a class of bug that would compile silently in C.

5.1 IsogenyDegree

Isogeny degrees in SQIsign are always positive odd integers bounded by 2^{f-2} (246 bits for NIST-I). In the C reference, they are bare `ibz_t` values—the same type used for everything else. Nothing prevents passing an even number or a negative value.

We represent degrees as `IsogenyDegree([u64; 4])`—unsigned, positive by construction, oddness enforced by the only constructor:

```
pub fn new_odd(limbs: [u64; 4]) -> Option<Self> {
    if limbs[0] & 1 == 0 { return None; }
    Some(Self(limbs))
}
```

Passing an even number to `FixedDegreeIsogeny` is a type error.

5.2 TorsionExponent

Torsion exponents (e such that 2^e divides the group order) are bounded by $f = 248$. A bare `u32` allows 4 billion values; only 249 are valid. `TorsionExponent(u32)` with a range check in the constructor catches this at the API boundary. Every function that takes a torsion exponent—`action_matrix`, `to_kernel`, `fixed_degree_isogeny`—uses this type, not `u32`.

5.3 HnfLattice vs. Lattice

Lattice containment checking requires Hermite Normal Form. Calling it on a non-HNF lattice silently produces wrong answers. We use two types: `Lattice<N>` (arbitrary basis) and `HnfLattice<N>` (guaranteed HNF, constructed only via `From<Lattice>`). The `contains()` method exists only on `HnfLattice`. You cannot call it on the wrong type.

5.4 IdealFactor

`SuitableIdeals` returns two factors, each with an element β, a degree d, and a parent order. In the C reference, these are six separate variables. Accidentally pairing one factor's degree with the other's element is a plausible mistake—the types are the same. Our `IdealFactor` struct bundles them:

```
pub struct IdealFactor {
    pub order: &'static ExtremalOrder<4>,
    pub beta: Element,
    pub degree: IsogenyDegree,
}
```

Cross-wiring requires reaching into both structs and swapping fields deliberately—not something you do by accident.

6 Constant-Time Considerations

6.1 The spec's silence on side channels

The SQIsign specification analyzes `SuitableIdeals` only in terms of failure probability (§9.3.2), not side-channel resistance. By tracing Algorithm 4.2 (signing), we confirmed that the quaternion algorithms operate on data derived from the secret key I_{sk} (lines 13–19). The variable-time behavior of the C reference is a pragmatic gap, not a deliberate security argument.

6.2 Published CT techniques

Seven papers form a complete constant-time stack for SQIsign v2:

- Kouider et al. [KMJK25] provide constant-time big integer arithmetic (division, exponentiation, square root) up to ∼12,000 bits.
- Hanyecz et al. [HKK+25] give the first constant-time LLL-like lattice reduction for dimension 4, replacing the variable-time L2 algorithm used in `SuitableIdeals`.
- Basso et al. [BHJ+25] cover CT HNF, CT `IdealGenerator`, and CT `RepresentInteger`—the three core quaternion subroutines.

- Kim et al. [KLKL25] prove explicit worst-case size bounds for all quaternion intermediates in Round-2 SQIsign, enabling fixed-precision (no multi-precision) arithmetic.
- Mukherjee et al. [MCJ+25] and Jacquemin et al. [JMKR23] demonstrate and propose mitigations for SPA attacks on Cornacchia's algorithm—the most urgent constant-time target.

We plan a six-phase CT hardening immediately after achieving end-to-end interoperability.

7 Current Status and Future Work

As of writing, 157 unit tests pass. The full `IdealToIsogeny` pipeline is wired end-to-end. KAT verification runs to completion but fails at the $(2, 2)$-chain splitting step due to remaining theta-coordinate bugs. The immediate next steps are:

1. Fix remaining theta bugs and achieve KAT verification;
2. Wire `SigningKey::sign()` and `generate()`;
3. Achieve full KAT interoperability;
4. Execute the six-phase CT hardening plan.

References

[AAC+24] Martha Norberg Hovd Aardal, Maria Pearson Azimova, Efrén López Carrión, Lorenz Dillinger, and Matthias J. Kannwischer. Optimized One-Dimensional SQIsign Verification on Intel and Cortex-M4. Cryptology ePrint Archive, Paper 2024/1563, 2024. https://eprint.iacr.org/2024/1563.

[BHJ+25] Andrea Basso, Chenfeng He, David Jacquemin, Fatna Kouider, Péter Kutas, Anisha Mukherjee, Sina Schaeffler, and Sujoy Sinha Roy. Constant-Time Quaternion Algorithms for SQIsign. Cryptology ePrint Archive, Paper 2025/2192, 2025. https://eprint.iacr.org/2025/2192.

[HKK+25] Otto Hanyecz, Alexander Karenin, Elena Kirshanova, Péter Kutas, and Sina Schaeffler. Constant Time Lattice Reduction in Dimension 4 with Application to SQIsign. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):511–534, 2025.

[JMKR23] David Jacquemin, Anisha Mukherjee, Péter Kutas, and Sujoy Sinha Roy. Ready to SQI? Safety First! Cryptology ePrint Archive, Paper 2023/807, 2023. https://eprint.iacr.org/2023/807.

[KLKL25] Won Kim, Jeonghwan Lee, Hyeonhak Kim, and Changmin Lee. SQIsign with Fixed-Precision Integer Arithmetic. Cryptology ePrint Archive, Paper 2025/1649, 2025. https://eprint.iacr.org/2025/1649.

[KMJK25] Fatna Kouider, Anisha Mukherjee, David Jacquemin, and Péter Kutas. Constant-Time Integer Arithmetic for SQIsign. Cryptology ePrint Archive, Paper 2025/832, 2025. https://eprint.iacr.org/2025/832.

[MCJ+25] Anisha Mukherjee, Maciej Czuprynko, David Jacquemin, Péter Kutas, and Sujoy Sinha Roy. Simple Power Analysis Attack on SQIsign. Cryptology ePrint Archive, Paper 2025/830, 2025. https://eprint.iacr.org/2025/830.

[Rus24] Rust team. Rust API Guidelines. https://rust-lang.github.io/api-guidelines/, 2024.

[SQI25] SQIsign team. SQIsign: Compact Post-Quantum Signatures from Quaternions and Isogenies. Specification version 2025-07-07, 2025. https://sqisign.org/spec/sqisign-20250707.pdf.